

```

0000 separate (IDL_SEM_PHASE)
package body PRA_WALK is
use REQ_UTIL; use DEF_UTIL; use EXP_TYPE; use EXPRESO; use SET_UTIL; use NOD_WALK; use PRAGMA_ERROR; use exception; use procedure WALK_PRAGMA
arguments (USED_NAME_ID, GEN_ASSOC_S : TREE; H : H_TYPE); use procedure GET_ARGUMENT_NAME (USED_NAME_ID : TREE; ASSOC_LIST : in out SEQ_TYPE; ASSOC_OU
T : out TREE; NEW_ASSOC_LIST : in out SEQ_TYPE; ARGUMENT_LIST : SEQ_TYPE); use procedure GET_ARGUMENT_EXP (USED_NAME_ID : TREE; ASSOC_LIST : in out SEQ
_TYPE; ASSOC_OUT : out TREE); use procedure MUST_BE_SIMPLE_NAME (EXP : TREE); use procedure MUST_BE_NAME (EXP : TREE); use procedure WALK_PRAGMA (USED_NA
ME_ID : TREE; GEN_ASSOC_S : TREE; H : H_TYPE) is
DEFINITE : SEQ_TYPE := LIST (D (LX_SYMPREP, USED_NAME_ID)); DEF : TREE; PRAGMA_DE
FN : TREE := TREE_VOID; begin
-- FIND THE PRAGMA_ID while not IS_EMPTY (DEFINITE) loop
D (XO_SOURCE_NAME, DEF); if PRAGMA_DEFN.TY = DN_PRAGMA_ID then exit; else PRAGMA_DEFN := TREE_VOID; end if; end
loop;
-- STORE THE PRAGMA_ID (OR VOID) D (SM_DEFN, USED_NAME_ID, PRAGMA_DEFN); -- IF PRAGMA_ID FOUND IF PRAGMA_DEFN /= TREE_V
0010 OID then
-- SUPPRESS FATAL ERRORS PRAGMA_CONTEXT := USED_NAME_ID; -- WALK THE ARGUMENTS WALK_PRAGMA_ARGUMENTS (USED_NAME_ID,
GEN_ASSOC_S, H); -- ENABLE FATAL-ERRORS PRAGMA_CONTEXT := TREE_VOID; -- IF-ERROR IN PRAGMA, PUT OUT IGNORED MESSAGE if D (S
M_DEFN, USED_NAME_ID) = TREE_VOID then raise PRAGMA_ERROR; end if; -- ELSE -- SINCE PRAGMA_ID NOT FOUND else -- PUT OU
T ERROR WARNING (D (LX_SRCPOS, USED_NAME_ID), "PRAGMA NOT KNOWN TO IMPLEMENTATION - " & PRINT_NAME (D (LX_SYMPREP, USED_NAME_ID))); end if;
exception
-- IN CASE OF-ERROR IN ARGUMENT EVALUATION when PRAGMA_ERROR => -- CLEAR THE PRAGMA_ID D (SM_DEFN, USED_NAME_ID,
TREE_VOID); -- ENABLE FATAL-ERRORS PRAGMA_CONTEXT := TREE_VOID; WARNING (D (LX_SRCPOS, USED_NAME_ID), "PRAGMA IGNORED - " & PRINT
NAME (D (LX_SYMPREP, USED_NAME_ID))); end WALK_PRAGMA_ARGUMENTS (USED_NAME_ID, GEN_ASSOC_S : TREE; H : H_TYPE) is PRAGM
A_ID : TREE := D (SM_DEFN, USED_NAME_ID); ARGUMENT_ID_LIST : SEQ_TYPE := LIST (D (SM_ARGUMENT_ID_S, PRAGMA_ID)); ASSOC_LIST : SEQ_TYPE := (TR
LIST (GEN_ASSOC_S); ASSOC_NODE : TREE; ASSOC_EXP : TREE; ASSOC_TYPE : TREE; DEFSET : DEFSET_TYPE; DEFINTERP : DEFINTERP_TYPE;
TYPESET : TYPESET_TYPE; DEF : TREE; ID : TREE; IDLIST : SEQ_TYPE := (TREE_NIL, TREE_NIL); NEW_ASSOC_LIST : SEQ_TYPE := (TR
EE_NIL, TREE_NIL); begin
case DEFINED_PRAGMAS_VAL (DI (XO_POS, D (SM_DEFN, USED_NAME_ID))) is
when CONTROLLED => -- $$$-IMMEDIATELY
Y WITHIN DECLARATIVE PART OR PACKAGE SPECIFICATION GET_ARGUMENT_EXP (USED_NAME_ID, ASSOC_LIST, ASSOC_EXP); MUST_BE_SIMPLE_NAME (ASSOC
EXP); ASSOC_EXP := WALK_TYPE_MARK (ASSOC_EXP); NEW_ASSOC_LIST := APPEND (NEW_ASSOC_LIST, ASSOC_EXP); ASSOC_TYPE := GET_B
ASE_STRUCT (ASSOC_EXP); if ASSOC_TYPE.TY /= DN_ACCESS or else D (SM_DERIVED, ASSOC_TYPE) /= TREE_VOID then ERROR (D (LX_SRCPOS, ASSOC_EXP),
CONTROLLED NOT ALLOWED FOR TYPE); else if D (XO_REGION, D (XO_SOURCE_NAME, ASSOC_TYPE)) /= D (XO_SOURCE_NAME, H.REGION_DEF) then ERROR (D (LX_
SRCPOS, ASSOC_EXP), "CONTROLLED NOT ALLOWED HERE"); else DB (SM_IS_CONTROLLED, ASSOC_TYPE, True); end if; when ELABORATE =>
-- $$$-ONLY AFTER CONTEXT CLAUSE; MUST BE IN CONTEXT CLAUSE loop GET_ARGUMENT_EXP (USED_NAME_ID, ASSOC_LIST, ASSOC_EXP); MUST_BE_SIMPLE
NAME (ASSOC_EXP); FIND_DIRECT_VISIBILITY (ASSOC_EXP, DEFSET); REQUIRE_UNIQUE_DEF (ASSOC_EXP, DEFSET); ASSOC_EXP := RESOLVE_NAME (ASSOC_EXP, GE
T_THE_ID (DEFSET)); NEW_ASSOC_LIST := APPEND (NEW_ASSOC_LIST, ASSOC_EXP); if D (SM_DEFN, ASSOC_EXP).TY not in CLASS_NON_TASK_NAME and D (SM_DEFN, ASSOC
_EXP) /= TREE_VOID then ERROR (D (LX_SRCPOS, ASSOC_EXP), "LIBRARY UNIT NAME REQUIRED"); end if; exit when IS_EMPTY (ASSOC_LIST); end l
oop;
when INLINE => -- $$$-AT PLACE OF DECLARATIVE ITEM OR FOLLOWING LIBRARY-UNIT loop GET_ARGUMENT_EXP (USED_NAME_ID, ASSOC_LIS
T, ASSOC_EXP); MUST_BE_SIMPLE_NAME (ASSOC_EXP); FIND_DIRECT_VISIBILITY (ASSOC_EXP, DEFSET); while not IS_EMPTY (DEFSET) loop POP (DEFSET, DEFINTE
R); DEF := GET_DEF (DEFINTERP); ID := D (XO_SOURCE_NAME, DEF); if D (XO_REGION_DEF, DEF) = H.REGION_DEF and then ID.TY in CLASS_SUBPROG_N
AME or else (ID.TY = DN_GENERIC_ID and then D (SM_SPEC, ID).TY in DN_PROCEDURE_SPEC .. DN_FUNCTION_SPEC) then IDLIST := APPEND (IDLIST, ID);
DB (SM_IS_INLINE, ID, True); end if; end loop; if IS_EMPTY (IDLIST) then ERROR (D (LX_SRCPOS, ASSOC_EXP), "NO SUCH SUBPROGRAM"); end if;
D (SM_DEFN, ASSOC_EXP, CAST_TREE (IDLIST)); ASSOC_EXP := MAKE_USED_NAME_ID_FROM_OBJECT (ASSOC_EXP); NEW_ASSOC_LIST := APPEND (NEW_ASSOC_LIST,
SSOC_EXP); exit when IS_EMPTY (ASSOC_LIST); end loop; when INTERPRET => -- $$$-AT PLACE OF DECLARATIVE ITEM OR FOLLOWING LIBRARY
-UNIT SPEC GET_ARGUMENT_NAME (USED_NAME_ID, ASSOC_LIST, ASSOC_EXP, NEW_ASSOC_LIST, ARGUMENT_ID_LIST); GET_ARGUMENT_EXP (USED_NAME_ID,
ASSOC_LIST, ASSOC_EXP); MUST_BE_NAME (ASSOC_EXP); FIND_VISIBILITY (ASSOC_EXP, DEFSET); while not IS_EMPTY (DEFSET) loop POP (
DEFSET, DEFINTERP); DEF := GET_DEF (DEFINTERP); ID := D (XO_SOURCE_NAME, DEF); if D (XO_REGION_DEF, DEF) = H.REGION_DEF and then ID.TY in CLASS_S
UBPROG_NAME then IDLIST := APPEND (IDLIST, ID); if D (XO_BODY, ID) = TREE_VOID or else D (XO_STUB, ID) = TREE_VOID then ERROR (D (LX_SRCPO
S, ASSOC_EXP), "BODY ALREADY GIVEN"); end if; end loop; if IS_EMPTY (IDLIST) then ERROR (D (LX_S
RCPOS, ASSOC_EXP), "NO SUCH SUBPROGRAM"); end if; D (SM_DEFN, ASSOC_EXP, CAST_TREE (IDLIST)); ASSOC_EXP := MAKE_USED_NAME_ID
FROM_OBJECT (ASSOC_EXP); NEW_ASSOC_LIST := APPEND (NEW_ASSOC_LIST, ASSOC_EXP); while not IS_EMPTY (IDLIST) loop POP (IDLIST, ID);
D (SM_INTERFACE, ID, D (SM_DEFN, HEAD (NEW_ASSOC_LIST))); end loop; when LIST => -- $$$-NOT GENERATING LISTING GET_ARGUME
NT_NAME (USED_NAME_ID, ASSOC_LIST, ASSOC_EXP, NEW_ASSOC_LIST, ARGUMENT_ID_LIST); when MEMORY_SIZE => ERROR (D (LX_SRCPOS, USED_NAME_ID),
"PRAGMA MEMORY_SIZE NOT SUPPORTED"); GET_ARGUMENT_EXP (USED_NAME_ID, ASSOC_LIST, ASSOC_EXP); when OPTIMIZE => GET_ARGUMENT_NA
ME (USED_NAME_ID, ASSOC_LIST, ASSOC_EXP, NEW_ASSOC_LIST, ARGUMENT_ID_LIST); when PACK => -- $$$-POSITIONS-AS FOR REPRESENTATION CLAUSE;-B
EFORE REP ATTR GET_ARGUMENT_EXP (USED_NAME_ID, ASSOC_LIST, ASSOC_EXP); MUST_BE_SIMPLE_NAME (ASSOC_EXP); ASSOC_EXP := WALK_T
YPE_MARK (ASSOC_EXP); NEW_ASSOC_LIST := APPEND (NEW_ASSOC_LIST, ASSOC_EXP); ASSOC_TYPE := GET_BASE_STRUCT (ASSOC_EXP); if
0050 SSOE_TYPE.TY not in DN_ARRAY .. DN_RECORD then ERROR (D (LX_SRCPOS, ASSOC_EXP), "PACK NOT ALLOWED FOR TYPE"); else if D (XO_REGION, D (XO_SOURC
E_NAME, ASSOC_TYPE)) /= D (XO_SOURCE_NAME, H.REGION_DEF) then ERROR (D (LX_SRCPOS, ASSOC_EXP), "PACK NOT ALLOWED HERE"); else DB (SM_IS_PACK
ED, ASSOC_TYPE, True); end if; when PAGE => null; when PRIORITY => -- $$$-TASK OR MAIN PROGRAM GET_ARGUME
NT_EXP (USED_NAME_ID, ASSOC_LIST, ASSOC_EXP); EVAL_EXP_TYPES (ASSOC_EXP, TYPESET); REQUIRE_TYPE (PREDEFINED_INTEGER, ASSOC_EXP, TYPESET
); ASSOC_EXP := RESOLVE_EXP (ASSOC_EXP, TYPESET); if GET_STATIC_VALUE (ASSOC_EXP) = TREE_VOID then ERROR (D (LX_SRCPOS, ASSOC_EXP), "
PRIORITY MUST BE STATIC"); end if; NEW_ASSOC_LIST := APPEND (NEW_ASSOC_LIST, ASSOC_EXP); when SHARED => -- $$$-SAME DECLA
RATIVE PART OR PACKAGE SPECIFICATION GET_ARGUMENT_EXP (USED_NAME_ID, ASSOC_LIST, ASSOC_EXP); MUST_BE_SIMPLE_NAME (ASSOC_EXP);
ASSOC_EXP := WALK_NAME (DN_VARIABLE_ID, ASSOC_EXP); NEW_ASSOC_LIST := APPEND (NEW_ASSOC_LIST, ASSOC_EXP); ASSOC_TYPE := GET_BASE
_STRUCT (ASSOC_EXP); if D (SM_DEFN, ASSOC_EXP) = TREE_VOID then null; else if ASSOC_TYPE.TY in CLASS_SCALAR or ASSOC_TYPE.TY = DN_ACCESS
S then if D (SM_RENAMES_OBJ, D (SM_DEFN, ASSOC_EXP)) = TREE_VOID then DB (SM_IS_SHARED, D (SM_DEFN, ASSOC_EXP), True); else ERROR (D (LX_SRCPO
S, ASSOC_EXP), "MAY NOT BE SHARED"); end if; else if ASSOC_TYPE = TREE_VOID then ERROR (D (LX_SRCPOS, ASSOC_EXP), "MUST BE SCALAR OR ACCESS T
YPE"); end if; when STORAGE_UNIT => ERROR (D (LX_SRCPOS, USED_NAME_ID), "PRAGMA STORAGE_UNIT NOT SUPPORTED"); GET_ARGUM
ENT_EXP (USED_NAME_ID, ASSOC_LIST, ASSOC_LIST, ASSOC_EXP); when SUPPRESS => -- $$$-IMMEDIATELY WITHIN DECL PART OR PACKAGE-SPEC GET_ARGUMENT
NAME (USED_NAME_ID, ASSOC_LIST, ASSOC_EXP, NEW_ASSOC_LIST, ARGUMENT_ID_LIST); if not IS_EMPTY (ASSOC_LIST) then POP (ASSOC_LIST, ASSOC
_NODE); if ASSOC_NODE.TY = DN_ASSOC then if D (LX_SYMPREP, D (AS_USED_NAME, ASSOC_NODE)) /= D (LX_SYMPREP, HEAD (ARGUMENT_ID_LIST)) then ERROR
(D (LX_SRCPOS, ASSOC_NODE), "SELECTOR MUST BE ON =>"); raise PRAGMA_ERROR; end if; D (SM_DEFN, D (AS_USED_NAME, ASSOC_NODE), HEAD (ARGUMENT
_ID_LIST)); ASSOC_EXP := D (AS_EXP, ASSOC_NODE); else ASSOC_EXP := ASSOC_NODE; end if; if ASSOC_EXP.TY = DN_STRING_LITERAL then ASSOC_EXP
:= MAKE_USED_OP_FROM_STRING (ASSOC_EXP); end if; MUST_BE_NAME (ASSOC_EXP); FIND_VISIBILITY (ASSOC_EXP, DEFSET); REQUIRE_UNIQUE_DEF (ASSOC_EXP, DEF
SET); ID := GET_THE_ID (DEFSET); ASSOC_EXP := RESOLVE_NAME (ASSOC_EXP, ID); if ID.TY in CLASS_OBJECT_NAME FIRST .. DN_GENERIC_ID and then ID.TY /=
DN_PACKAGE_ID then null; else if ID /= TREE_VOID then ERROR (D (LX_SRCPOS, ASSOC_EXP), "SUPPRESS NOT ALLOWED ON THIS"); end if; if ASSOC_NODE.T
Y = DN_ASSOC then D (AS_EXP, ASSOC_NODE, ASSOC_EXP); else ASSOC_NODE := ASSOC_EXP; end if; NEW_ASSOC_LIST := APPEND (NEW_ASSOC_LIST, ASSOC_N
DE); end if; when SYSTEM_NAME => ERROR (D (LX_SRCPOS, USED_NAME_ID), "PRAGMA SYSTEM_NAME NOT SUPPORTED"); GET_ARGUMENT
_EXP (USED_NAME_ID, ASSOC_LIST, ASSOC_EXP); when PRENAME_DEBUG => GET_ARGUMENT_NAME (USED_NAME_ID, ASSOC_LIST, ASSOC_EXP, NEW_ASSOC_LIST
, ARGUMENT_ID_LIST); case LIST_ARGUMENTS_VAL (DI (XO_POS, D (SM_DEFN, ASSOC_EXP))) is
when OFF => IDL_DEBUG := False; when ON => IDL_D
EBUG := True; end case; if not IS_EMPTY (ASSOC_LIST) then WARNING (D (LX_SRCPOS, USED_NAME_ID), "TOO MANY PRAGMA ARGU
MENTS"); raise PRAGMA_ERROR; end if; LIST (GEN_ASSOC_S, NEW_ASSOC_LIST); end WALK_PRAGMA_ARGUMENTS; use procedure GET_ARGUMENT_NAME (
USED_NAME_ID : TREE; ASSOC_LIST : in out SEQ_TYPE; ASSOC_OUT : out TREE; NEW_ASSOC_LIST : in out SEQ_TYPE; ARGUMENT_LIST : SEQ_TYPE) is
TEMP_ARGUM
ENT_LIST : SEQ_TYPE := ARGUMENT_LIST; ARGUMENT_ID : TREE; ACTUAL_SYM : TREE; ASSOC_EXP : TREE; begin
GET_ARGUMENT_EXP (USED
_NAME_ID, ASSOC_LIST, ASSOC_EXP); -- IF ASSOC_EXP.TY = DN_USED_OBJECT_ID THEN -- PUT_LINE ( "USED NAME ID : " & PRINT_NAME (D (LX_SYMPREP, USED_
NAME_ID)) ); -- PUT_LINE ( "ASSOC EXP : " & PRINT_NAME (D (LX_SYMPREP, ASSOC_EXP)) ); -- WARNING (D (LX_SRCPOS, USED_NAME_ID), "AR
GUMENT ID REQUIRED"); -- RAISE PRAGMA_ERROR; END IF; ACTUAL_SYM := D (LX_SYMPREP, ASSOC_EXP); ARGUMENT_ID := TREE_VOID; while n
ot IS_EMPTY (TEMP_ARGUMENT_LIST) loop
if D (LX_SYMPREP, HEAD (TEMP_ARGUMENT_LIST)) = ACTUAL_SYM then
ARGUMENT_ID := HEAD (TEMP_ARGUMENT_L
IST); exit; end if; TEMP_ARGUMENT_LIST := TAIL (TEMP_ARGUMENT_LIST); end loop; if ARGUMENT_ID = TREE_VOID then WARNI
NG (D (LX_SRCPOS, USED_NAME_ID), "ARGUMENT ID INVALID"); raise PRAGMA_ERROR; end if; D (SM_DEFN, ASSOC_EXP, ARGUMENT_ID); NEW_ASSOC
_LIST := APPEND (NEW_ASSOC_LIST, MAKE_USED_NAME_ID_FROM_OBJECT (ASSOC_EXP)); ASSOC_OUT := ASSOC_EXP; end GET_ARGUMENT_NAME; use procedure GET_AR
GUMENT_EXP (USED_NAME_ID : TREE; ASSOC_LIST : in out SEQ_TYPE; ASSOC_OUT : out TREE) is
ASSOC_EXP : TREE; begin
if IS_EMPTY (ASSOC_LIST) then
WARNING (D (LX_SRCPOS, USED_NAME_ID), "ARGUMENT REQUIRED"); raise PRAGMA_ERROR; end if; POP (ASSOC_LIST, ASSOC_EXP); if ASSOC_EXP.TY = DN_STRING_LITERAL then
ASSOC_EXP := MAKE_USED_OP_FROM_STRING (ASSOC_EXP); end if; ASSOC_OUT := ASSOC_EXP; end GET_ARGUMEN
T_EXP; use procedure MUST_BE_SIMPLE_NAME (EXP : TREE) is
begin
if EXP.TY = DN_USED_OBJECT_ID then
WARNING (D (LX_SRCPOS, EXP), "SIMPLE NAM
E REQUIRED"); raise PRAGMA_ERROR; end if; end MUST_BE_SIMPLE_NAME; use procedure MUST_BE_NAME (EXP : TREE) is
begin
if EXP.TY not in
CLASS DESIGNATOR and then EXP.TY /= DN_SELECTED then
WARNING (D (LX_SRCPOS, EXP), "NAME REQUIRED"); raise PRAGMA_ERROR; end if; end
MUST_BE_NAME;
end
PRA_WALK;
end

```